Building Archon: From Financial Documents to Controlled Records on Nebius Serverless AI

Automated extraction and classification, human review, document linking, and deterministic completeness checks for small-business finance

#NebiusServerlessChallenge | #ServerlessAI | #FinTech | #LLM

Small-business finance does not start with a dashboard. It starts with a folder full of documents that somebody must understand and enter correctly: purchase and sales invoices, expense receipts, payroll registers, bank confirmations, payslips, and supplier statements.

The operational questions are simple to ask and expensive to answer manually:

- What is this document, and where does it belong?
- Is it a supplier invoice or a sales invoice?
- Was every expected document actually received and recorded?
- Which documents describe the same financial event?
- Does a payment or collection have supporting evidence?
- For payroll, do the register, the bank confirmation, and the payslips agree?

Archon is built around that control loop. It turns uploaded financial documents into structured, classified, reviewable records; links documents that describe the same event; and runs explicit checks before producing a period financial view. The goal is to make financial entry and control faster, clearer, and auditable.

The current build proves that architecture with two bounded control paths: automated document processing with a human review gate, and payroll-event linking with deterministic validation. It also contains a supplier-statement reconciliation component for structured statement entries. General invoice-to-bank-payment matching, collections matching, duplicate-payment detection, and tax-remittance verification are the next extensions of the same model, not claims about what this version already does.

Architecture diagram: the complete component graph is also available in the public repository. The diagram separates the Jobs-based target architecture from the inline execution mode used by the live Endpoint while this tenant has zero CPU AI-Jobs quota.

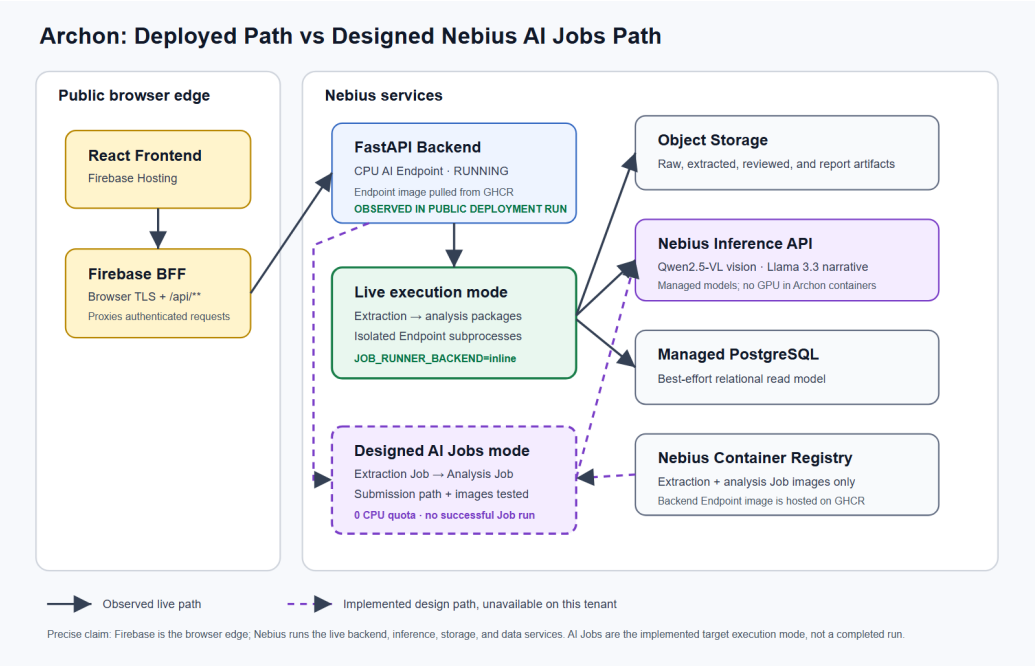

Figure 1. Archon spans a Firebase browser edge and Nebius backend services, with the current inline fallback shown explicitly.

The product loop: read, classify, review, record, control

Archon begins with mixed files rather than pre-cleaned rows. The extraction pipeline accepts PDFs, DOCX files, images, TIFFs, and scanned PDFs. Digital documents take the text path. Scanned and image-based documents go through Qwen2.5-VL-72B on the Nebius Inference API.

The result is a structured record with fields such as document type, date, supplier, recipient, tax identifier, currency, invoice number, VAT, totals, and line items. Payroll documents add purpose-specific fields such as employee count, gross pay, net pay, and employer cost.

Extraction is followed by deterministic classification. This second pass matters because an LLM can read a document correctly and still assign the wrong accounting type. `ClassifierAgent` refines ambiguous results using domain rules, keeping obvious classification errors out of downstream calculations.

The user then sees the successfully extracted documents before analysis. They can correct the type, exclude an unrelated file, and confirm the set that should proceed. Archon also checks whether a document appears to belong to the configured company by name or tax identifier.

There is an important current limitation around failed files. During extraction, Archon records each failed filename and reason inside the per-upload `documents.json` artifact in Object Storage and writes the failure to the job log. The current review API and UI do not expose that failure list, and confirming the reviewed set replaces the per-upload document artifact without carrying the failure metadata forward. Failures are therefore recorded during processing, but they are not yet visible to the reviewer in the product. Surfacing and preserving them through review is required before this can be described as a closed failure-handling loop.

That review gate is deliberate. Automation should remove repetitive entry work without removing control from the person responsible for the books.

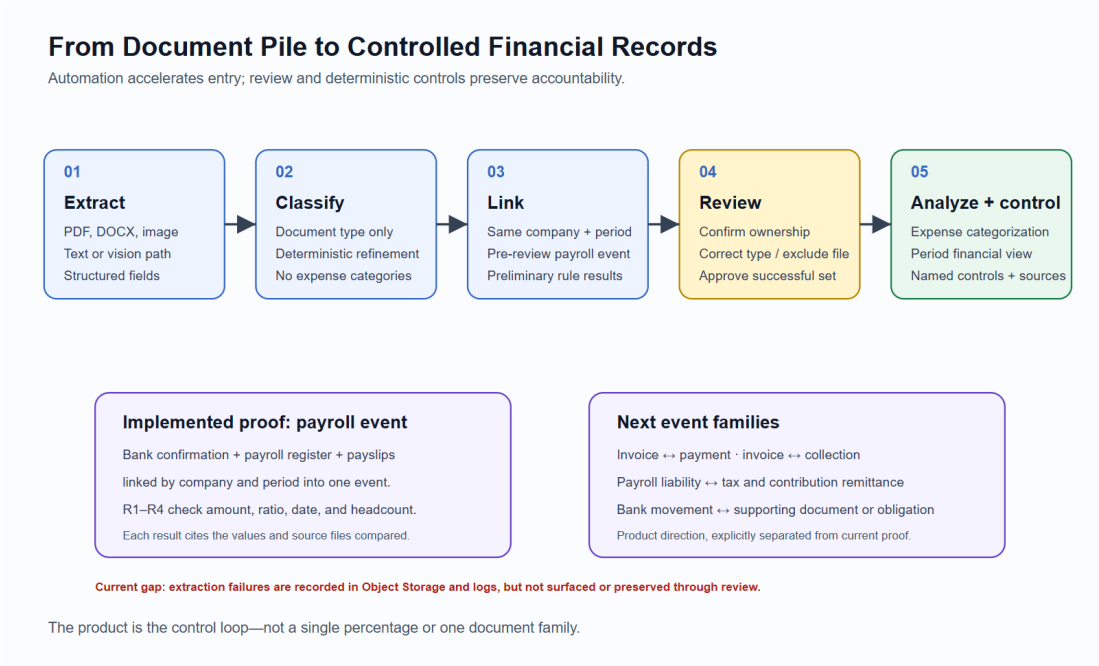

Figure 2. Archon keeps a human review gate between extraction and financial analysis.

```
# jobs/extraction/agents/classifier.py
def run(docs):
    for doc in docs:
        if doc.doc_type in (DocType.UNKNOWN, DocType.PAYROLL):
            doc.doc_type = _infer_type(doc)
    return docs
```

After approval, Object Storage holds the authoritative raw and structured artifacts. Managed PostgreSQL provides a relational read model for documents, payroll events, employees, and validation results. The database mirror is intentionally best-effort: a temporary database problem must not make an already-produced report disappear.

Linking documents that describe one event

Classification answers “what is this?” Linking answers “what does it belong with?”

Payroll is a useful worked example because a single payroll run produces several documents with different roles:

- The bank confirmation records the net amount transferred to employees.
- The payroll register records gross pay, employer contributions, employee count, and the full employer cost.
- Individual payslips explain the employee-level amounts.

These are complementary records for different parts of the same event, not competing versions of one number. Archon’s `EventLinkerAgent` groups them by company and period into a `PayrollEvent`. The cash-flow view reads the bank movement; the management expense view reads the register; validation checks whether the supporting records agree.

```
# jobs/extraction/agents/event_linker.py
def _build_event(company, period, docs):
    bank = _pick_one(docs, DocType.BANK_CONFIRMATION)
    register = _pick_one(docs, DocType.PAYROLL_REGISTER)
    payslips = [d for d in docs if d.doc_type == DocType.PAYSLIP]

    return PayrollEvent(
        period=period,
        company_name=company or None,
        bank_confirmation=bank,
        payroll_register=register,
```

```
        payslips=payslips,
        is_complete=bool(bank and register and payslips),
    )
```

Four named rules then check the linked evidence:

- R1: bank net approximately equals the sum of payslip nets, within ±2%.
- R2: employer cost divided by net pay falls inside an explicit expected band.
- R3: the bank-confirmation date is not later than the end of the payroll period.
- R4: register headcount equals the number of payslips.

Each result cites the rule, the compared values, and the source files. The output is not “the AI thinks something looks suspicious.” It is a control that a reviewer can reproduce by hand.

The broader product direction follows the same pattern. A supplier invoice should connect to its settlement evidence. A sales invoice should connect to its collection. A bank movement should be explainable by a document or obligation. Taxes and social-security liabilities should connect to their remittances. Those links are the natural next event families; the submitted build does not pretend they are already complete.

Supplier completeness: the precise current boundary

Archon includes a unit-tested `ReconciliationAgent` that compares pre-structured entries from a supplier statement with the invoice numbers and totals present in the system. Given those fields, it can report statement invoices that are missing from the uploaded set, uploaded invoices absent from the statement, and a balance discrepancy.

That is a document-completeness component, not yet a bank-payment matcher. It is invoked by the analysis pipeline when structured statement data is present, but the current extraction prompt does not request `statement_entries`, `statement_balance`, or `statement_overdue`, and the review UI does not collect them. The component is therefore tested at the analysis boundary but is not wired end to end from a raw supplier statement through extraction and review. “This bank payment settled that invoice” is separate roadmap work.

This distinction is also why Archon keeps supplier statements out of P&L and cash-flow arithmetic. A statement is reference evidence. Counting it as an expense would duplicate the invoices it lists.

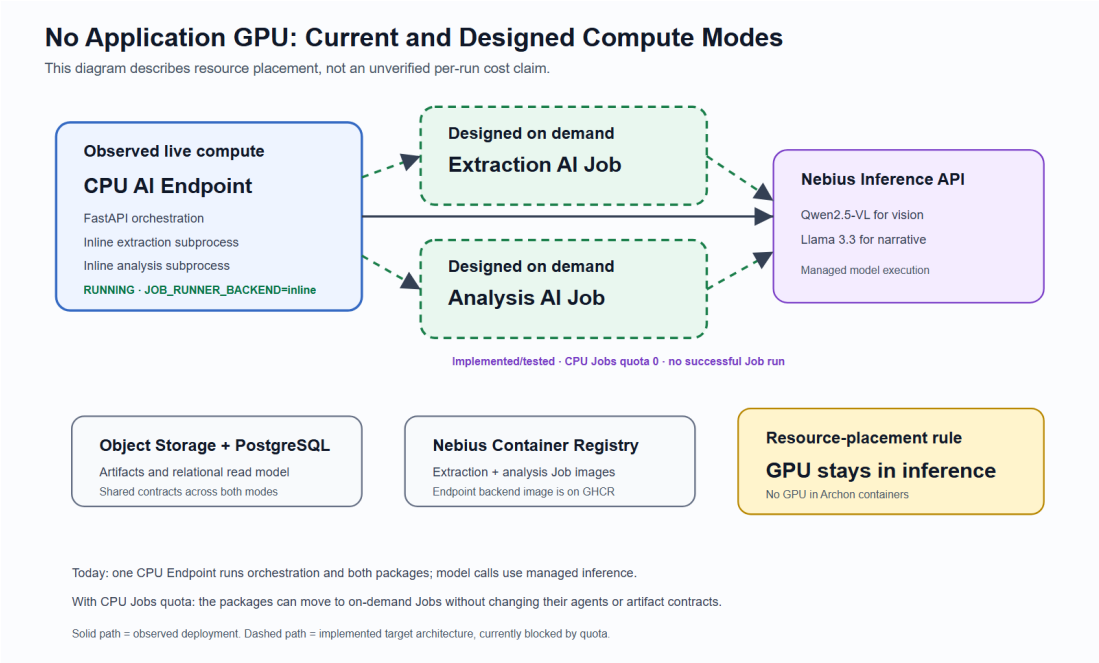

Figure 4. The live build uses a CPU Endpoint and managed inference; the AI Jobs path remains the burst-compute design.

Why Nebius Serverless AI fits the workflow

Financial-document processing is bursty. A business may upload a monthly batch, process it, inspect the results, and then do nothing for days or weeks. Keeping a dedicated GPU online for that pattern would be wasteful.

Archon separates orchestration from batch work:

- A CPU AI Endpoint hosts the FastAPI backend and the upload, review, job-status, analysis, and report APIs.
- An extraction AI Job is the designed on-demand path for processing an uploaded batch and writing structured artifacts.
- An analysis AI Job is the designed on-demand path for reading approved records, running the financial agents, and writing the report.
- The Nebius Inference API serves Qwen2.5-VL-72B for vision extraction and Llama-3.3-70B for the executive narrative.
- Object Storage and Managed PostgreSQL provide durable artifacts and a relational read model.
- Nebius Container Registry holds the extraction and analysis Job images. The Endpoint backend image is pulled from GitHub Container Registry.

The GPU lives in the managed inference layer rather than in Archon's containers. The extraction and analysis packages remain CPU-only whether they run as Jobs or through the fallback below.

The live deployment currently uses `JOB_RUNNER_BACKEND=inline`: the same two packages run as isolated subprocesses inside the CPU Endpoint and use the same Object Storage and status contracts. The Jobs submission code, images, and capacity handling are implemented and tested, but this tenant's zero CPU AI-Jobs quota means no Job has provisioned successfully. "Designed as AI Jobs" and "currently running inline" are intentionally separate claims.

The React frontend and a thin BFF run on Firebase for public hosting, authentication, and browser-edge TLS. The precise deployment claim is therefore: Nebius runs the domain backend, job design, inference, storage, registry, and financial data services; Firebase provides the public browser edge.

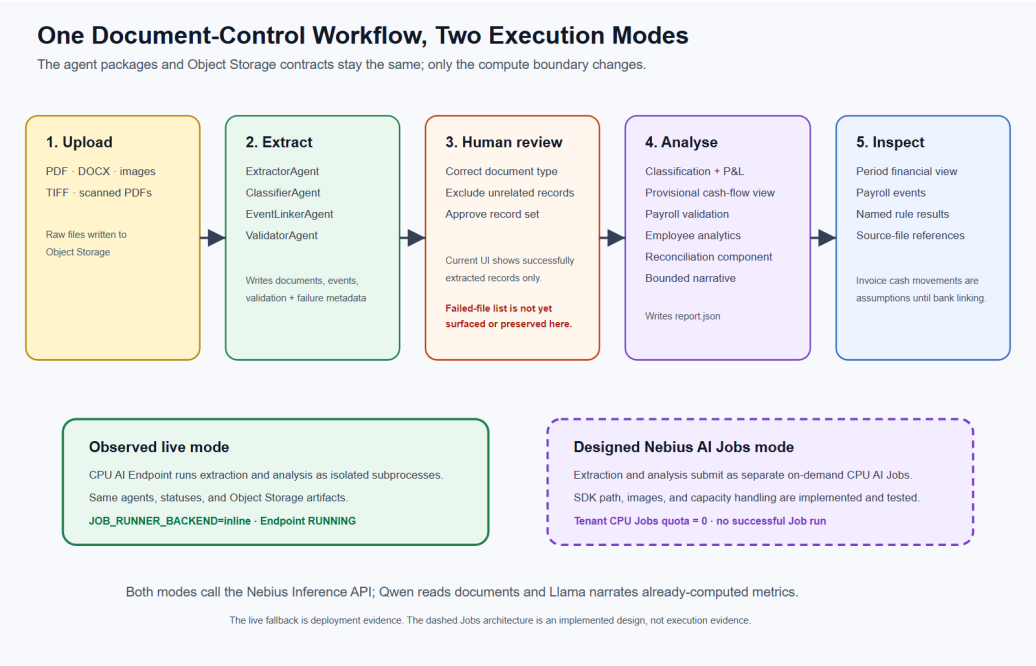

Figure 3. Both pipelines share artifact and status contracts across the designed Jobs path and the live inline fallback.

Two single-responsibility pipelines

The extraction package has four stages:

1. `ExtractorAgent` routes each file to text or vision extraction and emits structured fields.
2. `ClassifierAgent` refines the document type deterministically.
3. `EventLinkerAgent` groups documents that describe the same payroll event.
4. `ValidatorAgent` applies the named cross-document rules.

The analysis package has seven stages: classification, P&L aggregation, cash-flow construction, validation, employee analytics, supplier-statement reconciliation, and narrative generation.

These are the same packages in both execution modes. With Jobs capacity they are submitted as two on-demand Nebius AI Jobs. In the live submission they run as isolated subprocesses inside the Nebius AI Endpoint. The execution boundary changes; the agents and artifact contracts do not.

Small agents are not cosmetic. They make each responsibility independently testable. A failed extraction remains an extraction problem; a classification error does not become an unexplained reporting error; and a validation rule can be measured separately from the figures it checks.

Deterministic accounting, bounded model use

Archon does not ask a language model to calculate the financial totals. P&L figures and validation results are produced by Python arithmetic and explicit rules. The model reads messy documents and writes a narrative from already-computed metrics.

```
def build_pnl(period, docs):
    revenue = sum(d.total_amount for d in docs if d.doc_type in REVENUE_DOC_TYPES)
    expenses = _compute_expenses(docs)
    return MonthlyPnL(
        period=period,
        revenue=round(revenue, 2),
        expenses=round(expenses, 2),
        netProfit=round(revenue - expenses, 2),
    )
```

If narrative generation fails, the report still exists. The language model is useful at the unstructured edges of the workflow; it is not the ledger.

The current cash-flow output is a provisional document-derived view, not a bank-reconciled cash statement. Payroll cash uses the actual `bank_confirmation` transfer, but sales invoices are assumed collected and purchase invoices or expense documents are assumed paid. Until general payment and collection linking is implemented, those invoice-derived inflows and outflows must not be presented as verified bank movements.

Measuring the implemented controls

The repository includes an offline evaluation harness built around 40 labelled synthetic payroll cases. It imports the real `ClassifierAgent`, `EventLinkerAgent`, `ValidatorAgent`, and `PnLAgent` rather than reimplementing them inside the test.

Under a deterministic perfect-extraction ceiling, classification, selected-field accuracy, and payroll-fusion accuracy reach 100%. A deliberately degraded extractor drops classification to 74.29%, field accuracy to 77.62%, and fusion accuracy to 54.05%. That drop is useful: small field errors compound when records are linked.

The 100% figure is not a claim that live Qwen extraction is perfect. It is a ceiling test for the downstream agents given correct structured fields. Keeping that distinction explicit makes the benchmark useful rather than promotional.

The harness also found a real defect. R2 and R4 initially fired 0 out of 37 applicable cases because the extraction prompt did not request the register fields those rules consumed. After the fields were added and mapped, the same tests measured 37 out of 37. That before-and-after result is exactly what an evaluation harness should produce: evidence that a control is active, not just code that looks plausible.

```
python eval/generate_corpus.py --out corpus/full --n 40 --seed 7
python eval/evaluate.py --corpus eval/corpus/full --out eval/RESULTS_full.json
```

The benchmark runs offline with no API key and only `pydantic`. The public repository includes the generated results, tests, and reproduction commands.

An operational lesson from AI Jobs

The most useful Serverless engineering lesson came from a failure mode. A Nebius AI Job can be accepted while never receiving an instance when the selected compute preset has no available quota. The submission returns an ID, but the job remains in provisioning and eventually disappears.

Archon wraps job submission in a bounded capacity probe. It distinguishes:

1. submission rejected for capacity,
2. accepted but never provisioned,
3. an application that reached compute and then failed.

A capacity rejection, or a terminal/vanished Job that provably never received an instance, moves to the next configured preset. If an accepted Job is still provisioning when the observation window ends, Archon keeps and returns that pending Job rather than deleting it or creating a duplicate. A Job that reached compute and then failed is surfaced as an application failure, not retried on another preset.

This tenant currently has zero CPU AI-Jobs quota. The Jobs integration, SDK submission path, images, and capacity probe are built and tested, but no CPU Job can provision on the tenant. For the live product, `JOB_RUNNER_BACKEND=inline` runs the same extraction and analysis packages as isolated subprocesses inside the CPU Endpoint while preserving the same status and Object Storage contracts. It is a resilience path, not evidence of a successful Job run.

That disclosure matters. Reproducible engineering includes the limits of the environment in which it was tested.

Try the build

The public repository is MIT licensed. A fresh local run needs Docker, Python, `curl`, `jq`, and a Nebius Inference API key. First generate the synthetic PDFs and start the stack:

```
git clone https://github.com/upgradedev/archon_nebius
cd archon_nebius
cp .env.example .env
# Edit .env and replace the NEBIUS_INFERENCE_API_KEY placeholder.
python -m pip install reportlab
python scripts/generate-sample-data.py
docker compose up --build
```

Then, in a second terminal while the stack is running:

```
cd archon_nebius
bash scripts/test-pipeline.sh
```

The live demo renders an illustrative seeded financial view without authentication. Its internally consistent seeded records demonstrate the review and reporting UI; they are not evidence of bank matching, collection matching, or remittance verification. The public Nebius deployment run is infrastructure evidence: it records `archon-backend-r130` reaching `RUNNING`, a successful Object Storage round trip, Managed PostgreSQL reachable from the Endpoint, and the Firebase BFF health route returning HTTP 200. It does not claim that an AI Job ran; the live processing mode remains the disclosed inline fallback.

Archon's direction is straightforward: every successfully extracted document becomes an understandable record; every record has a category and an owner; related records form one financial event; and every validation result identifies the values and source files it compared. That is the foundation for answering the questions a business actually asks - what is this, why was it paid, what is still missing, and does the close reconcile?

Built for the Nebius Serverless AI Builders Challenge 2026. Code: https://github.com/upgradedev/archon_nebius (MIT).